



Université d'Ottawa • University of Ottawa

SEG3503 ASSURANCE DE LA QUALITÉ LOGICIELLE

Été 2026

Devoir 2

Date d'échéance: le 5 juillet à 20h

Le devoir est à faire INDIVIDUELLEMENT ou en équipes de deux étudiants MAXIMUM. Vous devez remettre une archive .ZIP contenant vos réponses aux questions ainsi que votre implémentation JUnit.

Problème 1

Le but de ce problème est de développer une suite de tests qui atteint **une couverture de 100% des branches** des méthodes de la classe **ISBNValidate** (celle du devoir 1).

Question 1.1 (25%)

Écrivez une suite de tests qui atteint une couverture de 100% des branches de chacune des méthodes de la classe **ISBNValidate**. Votre suite de tests sera évaluée sur le nombre de cas de test qu'elle contient (essayez d'avoir le plus petit nombre possible qui permettra une couverture à 100% des branches).

Fournissez une table qui montre les liens entre vos cas de test, vos données de test et les branches couvertes. Cette table doit avoir le format suivant :

Numéro du cas de test	Données de test avec résultat attendu	Branches couvertes

Vous pouvez identifier les branches avec les numéros de ligne du programme. Alternativement, vous pouvez inclure un graphe de flot de contrôle généré manuellement ou à l'aide d'un outil tel que **Control Flow Graph Factory** (<http://www.drgarbage.com/control-flow-graph-factory.html>) et référer à ce graphe pour identifier les branches.

Question 1.2 (15%)

Implémentez votre suite de tests en utilisant le framework JUnit. ***Vous devez remettre le code source de tous vos tests ainsi que tout autre code qui les supportent.***

Question 1.3 (10%)

Rapportez vos résultats de test:

- Quel est le degré de couverture des branches obtenu? Si vous n’avez pas pu atteindre 100%, expliquez pourquoi.
- Quels cas de test ont échoué? Comment?

Fournissez une table similaire à celle de la question 1.1, mais avec une colonne additionnelle pour les résultats actuels.

Attachez des captures d’écran montrant vos résultats de couverture du code.

Problème 2

Le but de ce problème est de développer une suite de tests basée sur l’approche N+. La machine d’état suivante modèle un système de E-booking utilisé dans les aéroports pour l’auto-enregistrement des passagers.

Ebook.zip contient une implémentation de la machine à état du système de E-booking (la classe **EbookingControl**) avec un simulateur de base.

Question 2.1 (10%)

Dressez un arbre des chemins aller-retour à partir du modèle d’état du système de E-booking.

Question 2.2 (15%)

Fournissez des suites de tests de conformité et des “sneak paths” pour le système de E-booking. Chaque cas de test doit avoir :

- Un identificateur
- Une description de la préparation du test (par exemple les objets nécessaires au cas de test)
- Une séquence de test en termes d’état de départ, d’évènement, de condition de garde / action attendue et de réponse attendue du système (nouvel état).

Question 3 (20%)

Implémentez votre suite de tests en utilisant le framework JUnit. ***Vous devez remettre le code source de tous vos tests ainsi que tout autre code qui les supportent.***

Question 4 (5%)

Résultats du test basé sur les états.

- Pour chaque cas de test
- Si l’exécution actuelle correspond à celle décrite dans le cas de test, indiquez que le test est un succès.
- Si l’exécution actuelle ne correspond pas à celle décrite dans le cas de test, donnez le résultat actuel et montrez où il y a contradiction.